

Inter-Process Communication

1) System V and POSIX interfaces

2) ~~Primitives~~ Primitives

→ ^(a) Pipes and FIFOs

→ ^(b) Message Queues

→ ^(c) Sockets

→ Datagramms

(Message-like structure)

→ Stream Sockets

Datagramms do not guarantee the delivery of packets in the right sequence.
For datagramms, read/write cannot be

used as they inherently depend on
sending/receiving streams of data.

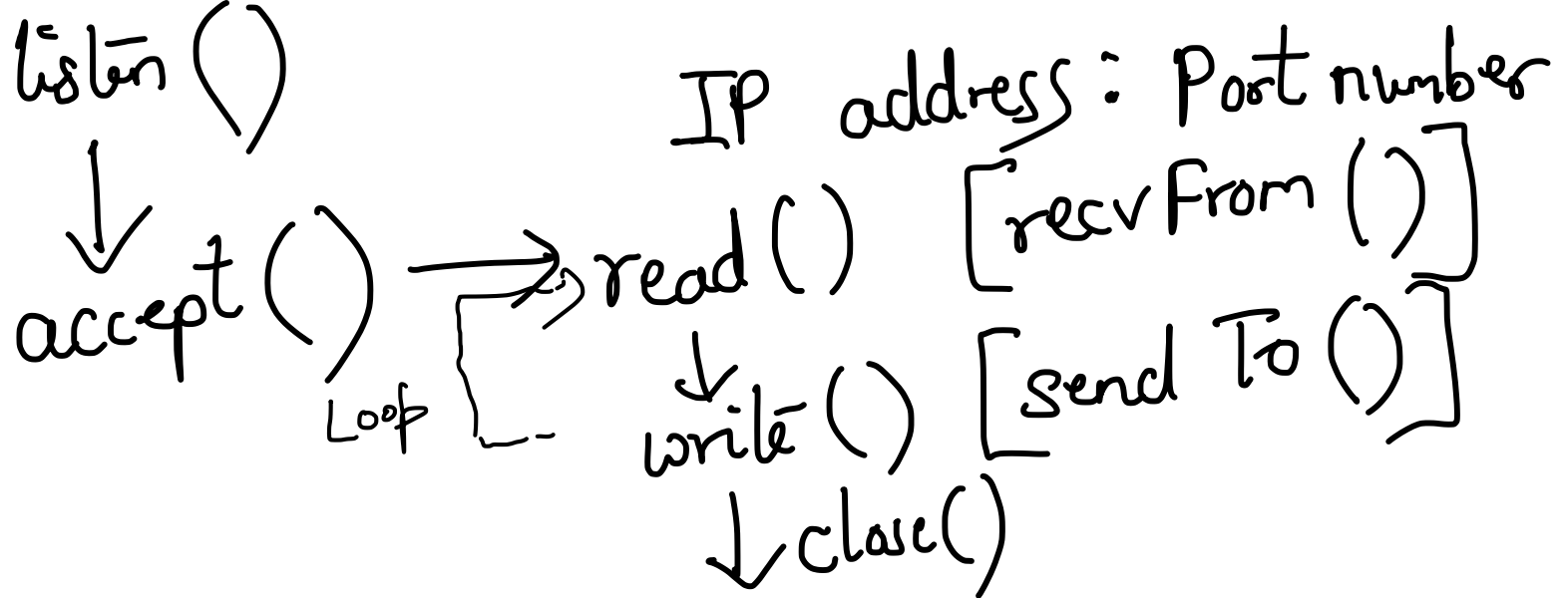
So, there are two new functions
that can be ~~so~~ used - `sendTo()`
and `recvFrom()`.

Server socket after its creation attaches
itself to a specific port number
and then listens for any
connection request.

Socket()

↓
bind (port number)
↓

What does it identify?
⇒ Identifies a
socket

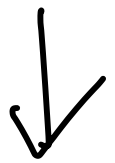


Communicating between multiple processes or threads within a machine → just port number is enough.

Communicating between processes across machines → IP address & port number

Client side

socket()



connect()



→ write() [sendTo()]

```

Loop {
    ↓
    read() [recv From()]
    ↓
    close()

```

Enforcing Sequence over a network has its performance overhead. It depends on the use case whether to use datagrams or streams.

(d) Shared Memory

Create a memory segment and share it among multiple processes.

```
a = shm_open("abc", RD_WR);
```

```
addr = mmap(NULL, a, ...);
```

/* Gives a virtual address to which you can write to share data using the shared memory segment */

```
memcpy("abcdefg", addr); // memory
addr[5] ~> *(addr + 5) (u_f 2345678, addr);
```

strcpy $\rightarrow$ requires your string to end with
& '\0' (null character)

Shared memory has the best performance.

$\Rightarrow$ Responsibility of ensuring that the data
is consistent is on the programs.

abcdefgh
12345678
1234 efgh
abcd1234

} Need to avoid these cases
using process synchronization
primitives Locks, Semaphores
(in the next week) can
be used to handle these
problems. The challenge of
using them is that the user
program needs to keep
track of how ^{other} programs can
use the shared data structure.

